



◆ CS EDUCATION ◆

Q

CORS



25.01.10
채다희



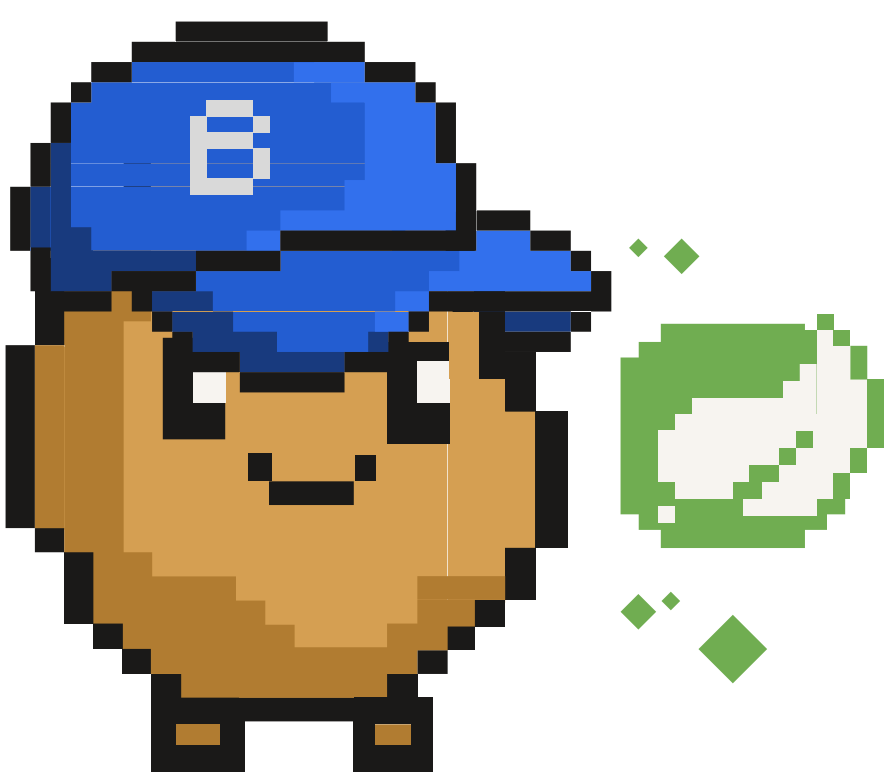
목차

1. 왜 발생하는 에러인가?
2. 어떻게 해결할 수 있을까?

http://localhost:3000



http://localhost:44300



POST

http://localhost:44300/api/route

교차 출처 자원 공유 Cross Origin Resource Sharing CORS

Access to XMLHttpRequest at 'https://localhost:44300/api/route/' from origin 'http://localhost:3000' has been blocked by CORS policy: Response to preflight request doesn't pass access control check: No 'Access-Control-Allow-Origin' header is present on the requested resource.

출처 Origin

https://www.google.com:80/search?q=cotato#first

프로토콜

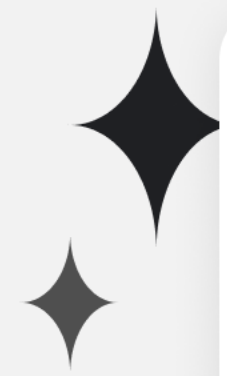
호스트

포트

경로

쿼리

프래그먼트



출처 Origin

https://www.google.com:80/search?q=cotato#first

프로토콜

호스트

포트

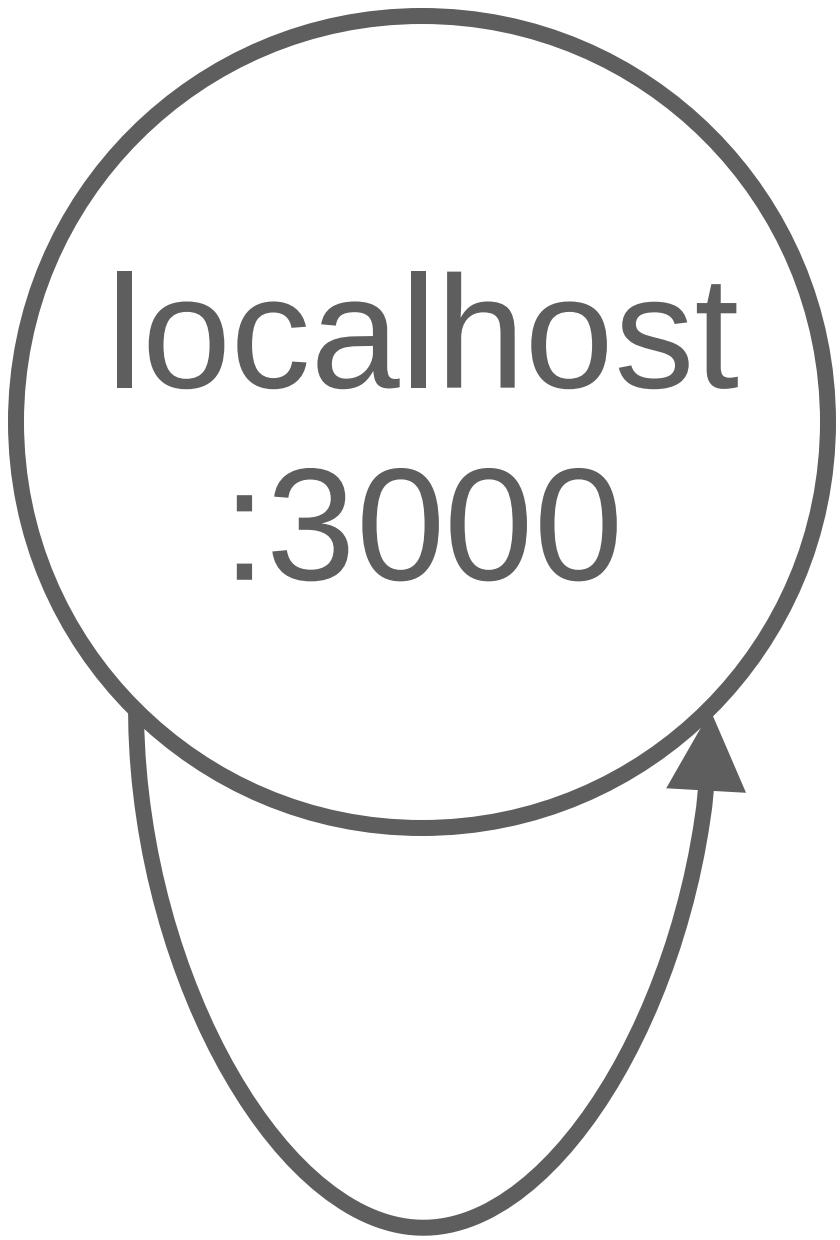
경로

쿼리

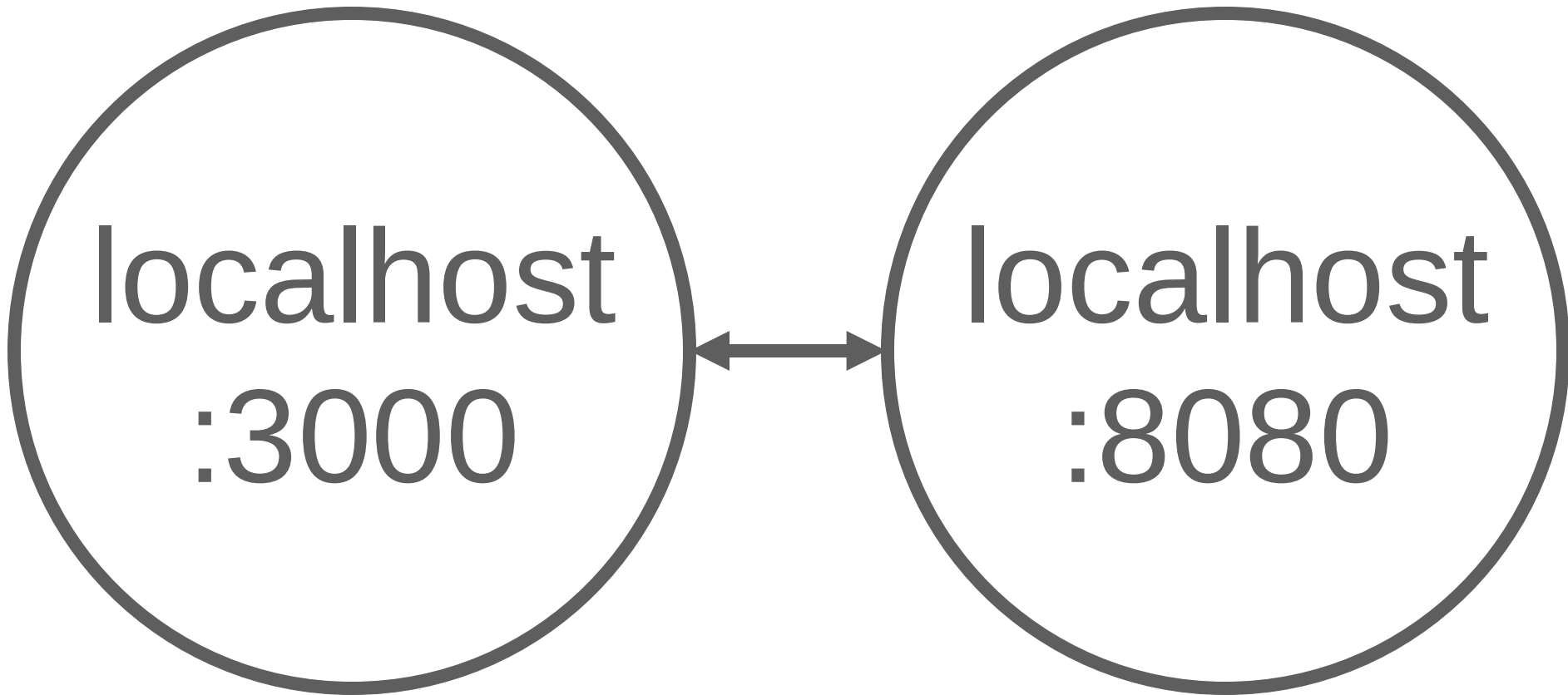
프래그먼트

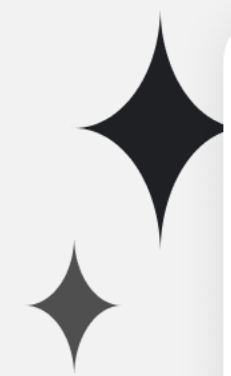


Same Origin Policy SOP



Cross Origin Resource Sharing CORS

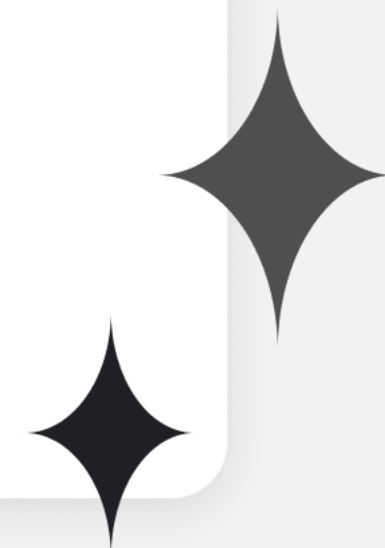


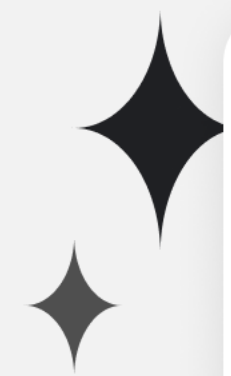


동일 출처 정책 SOP

Same Origin Policy
동일 출처 자원 공유
즉, 다른 출처 자원 공유 차단 ⚠️

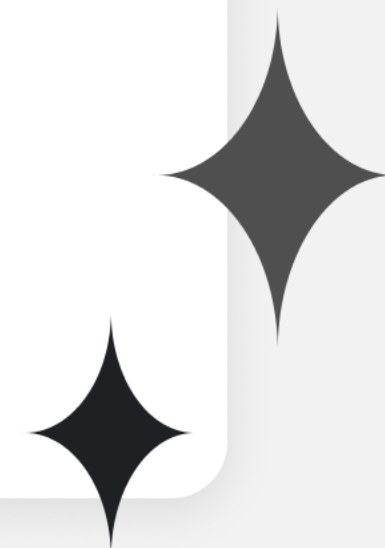
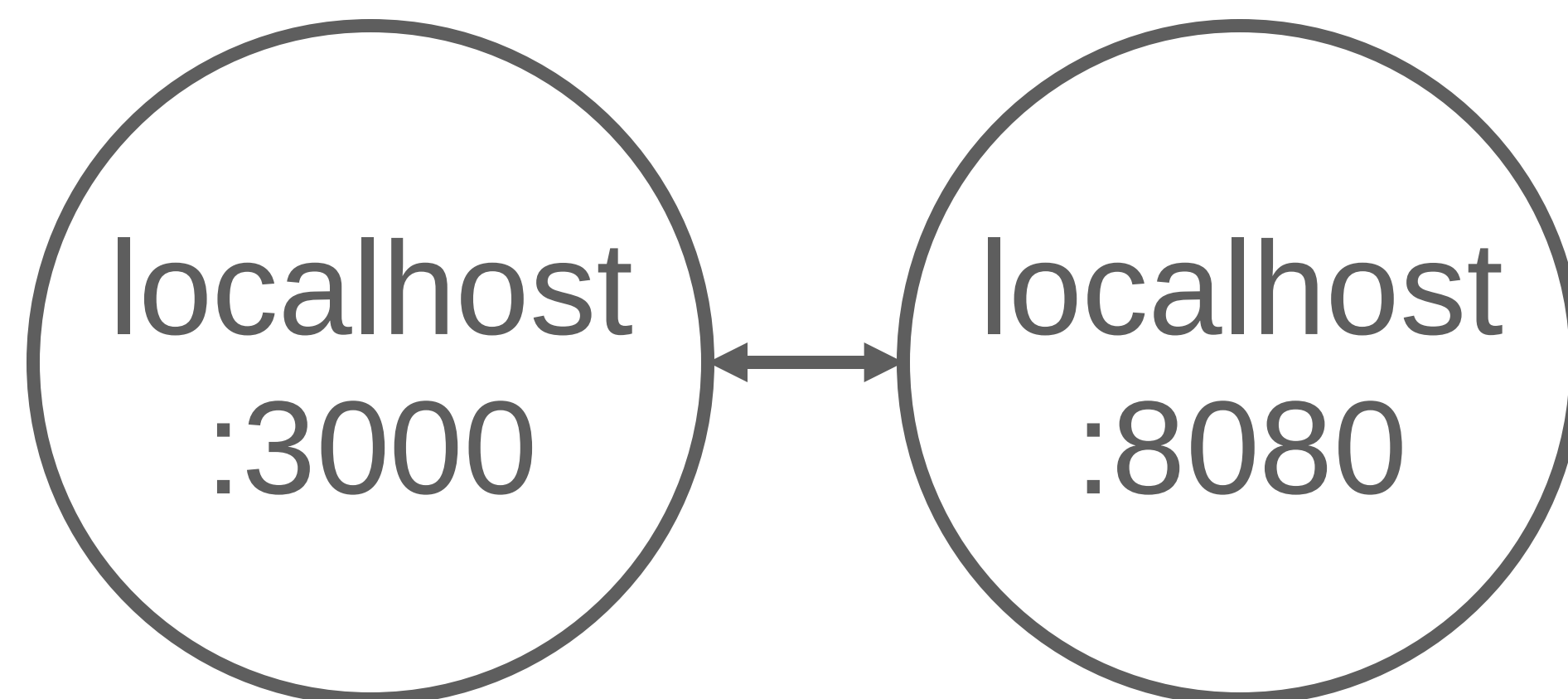
보안 필요성
CSRF, XSS 공격으로부터 보호



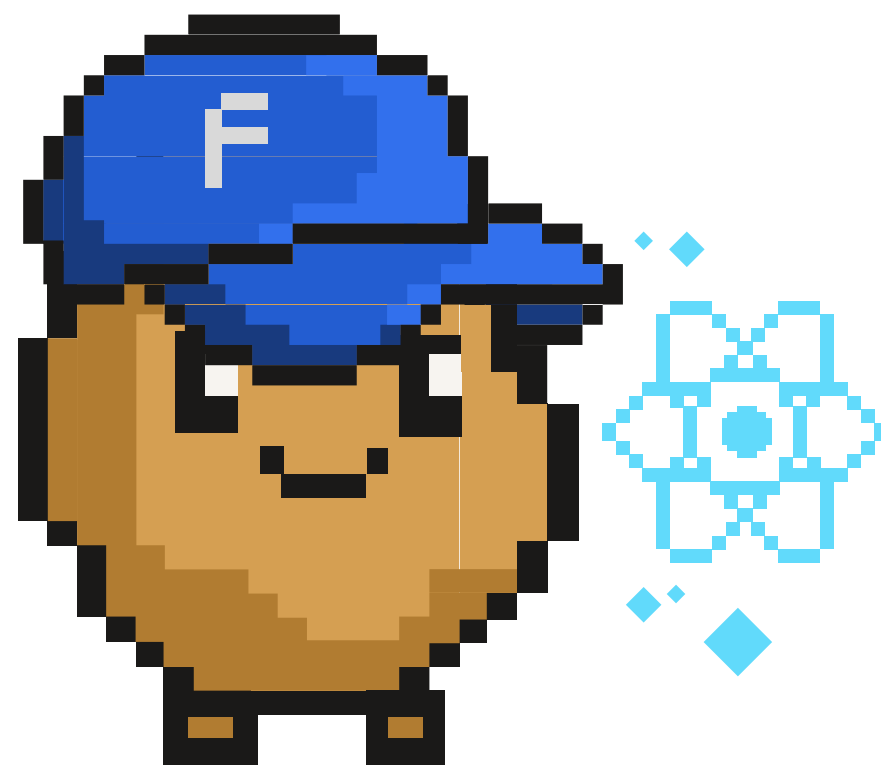


교차 출처 자원 공유 CORS

Cross Origin Resource Sharing
다른 출처 자원 공유 허용



Access to XMLHttpRequest at 'https://localhost:44300/api/route/' from origin 'http://localhost:3000' has been blocked by CORS policy: Response to preflight request doesn't pass access control check: No 'Access-Control-Allow-Origin' header is present on the requested resource.

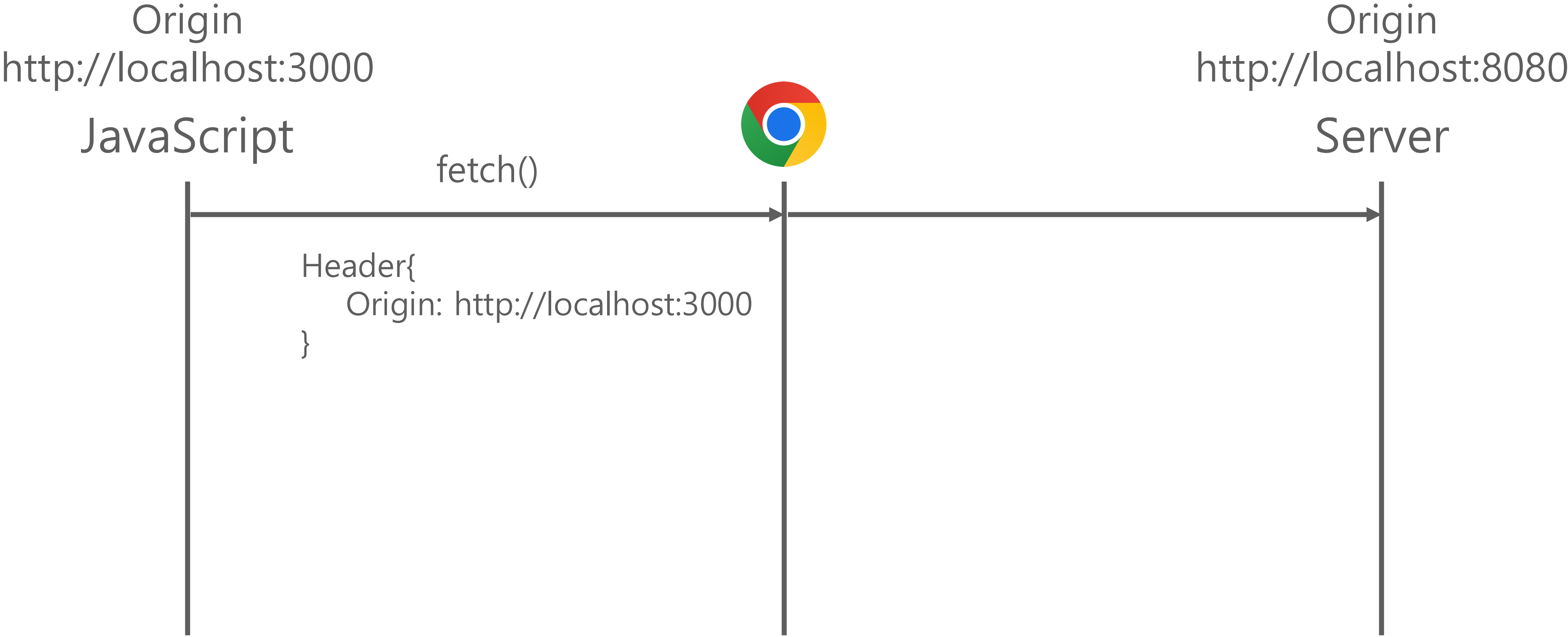


Proxy 서버

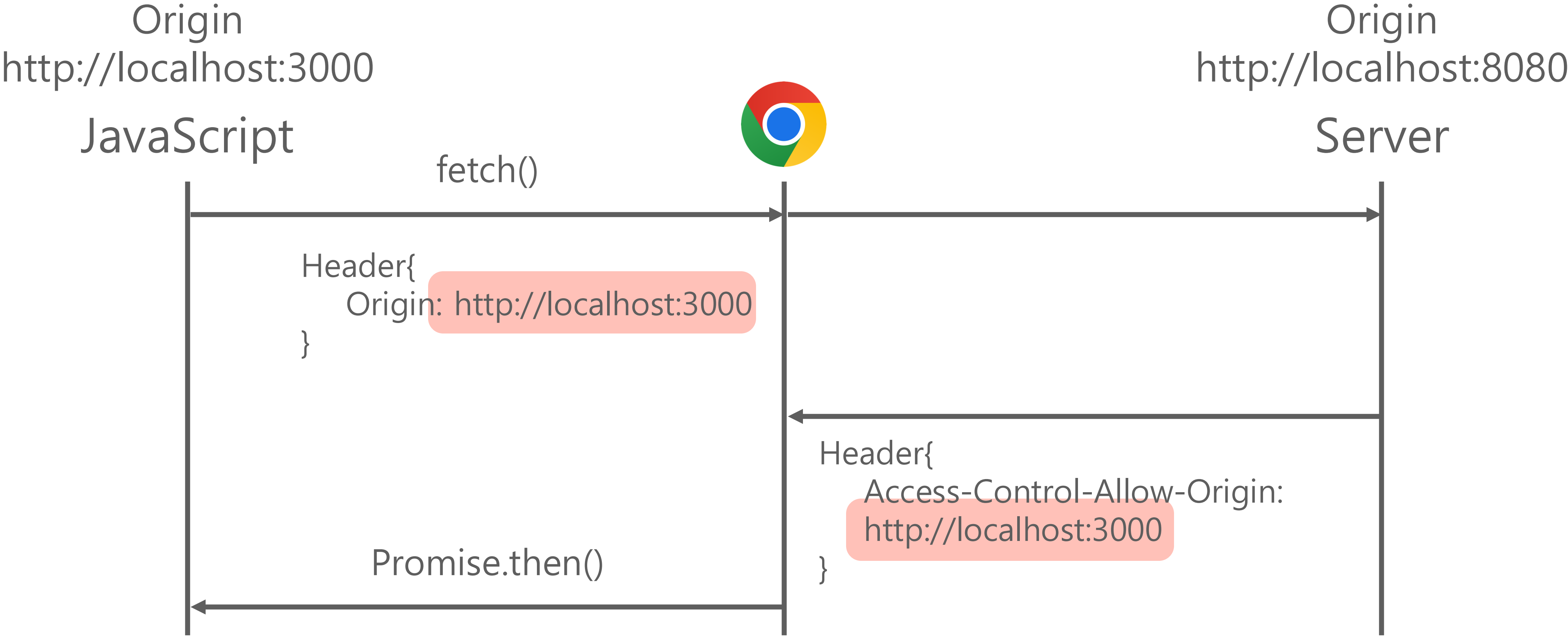


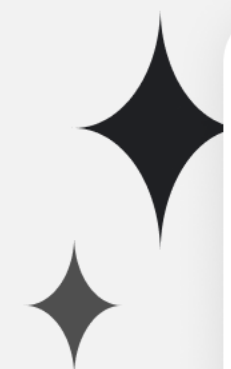
CORS 응답 헤더 설정

CORS 기본동작



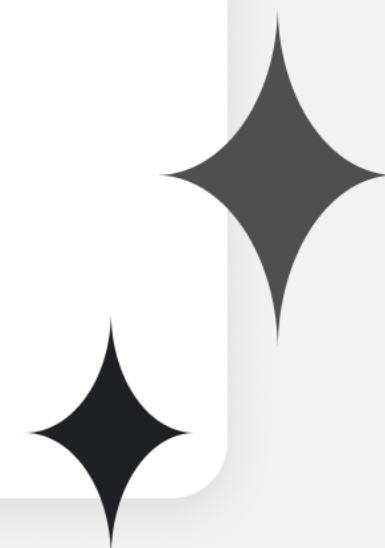
CORS 기본동작





CORS 시나리오

1. Simple Request 단순 요청
2. Preflight Request 예비 요청
3. Credentialed Request 인증된 요청



JavaScript



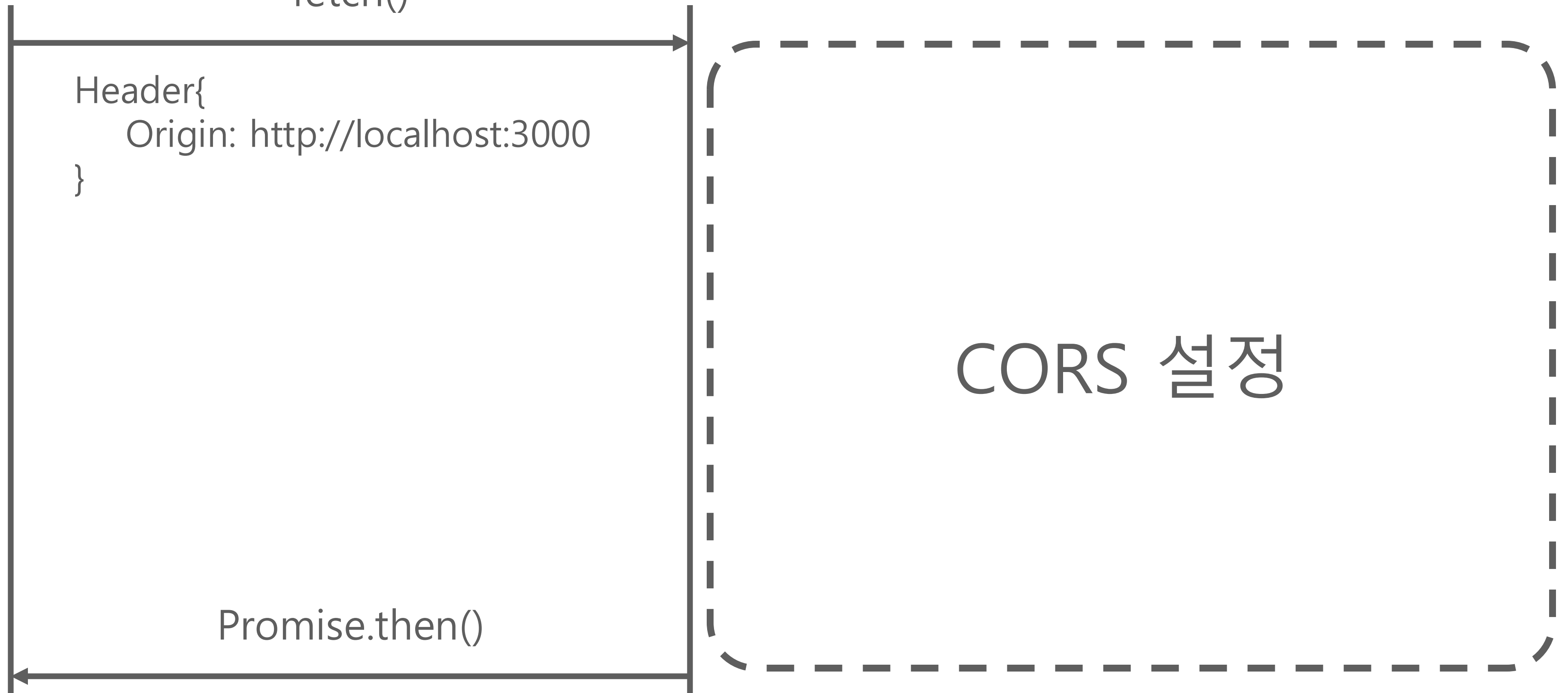
Server

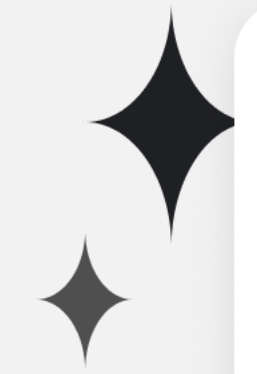
fetch()

```
Header{  
  Origin: http://localhost:3000  
}
```

CORS 설정

Promise.then()

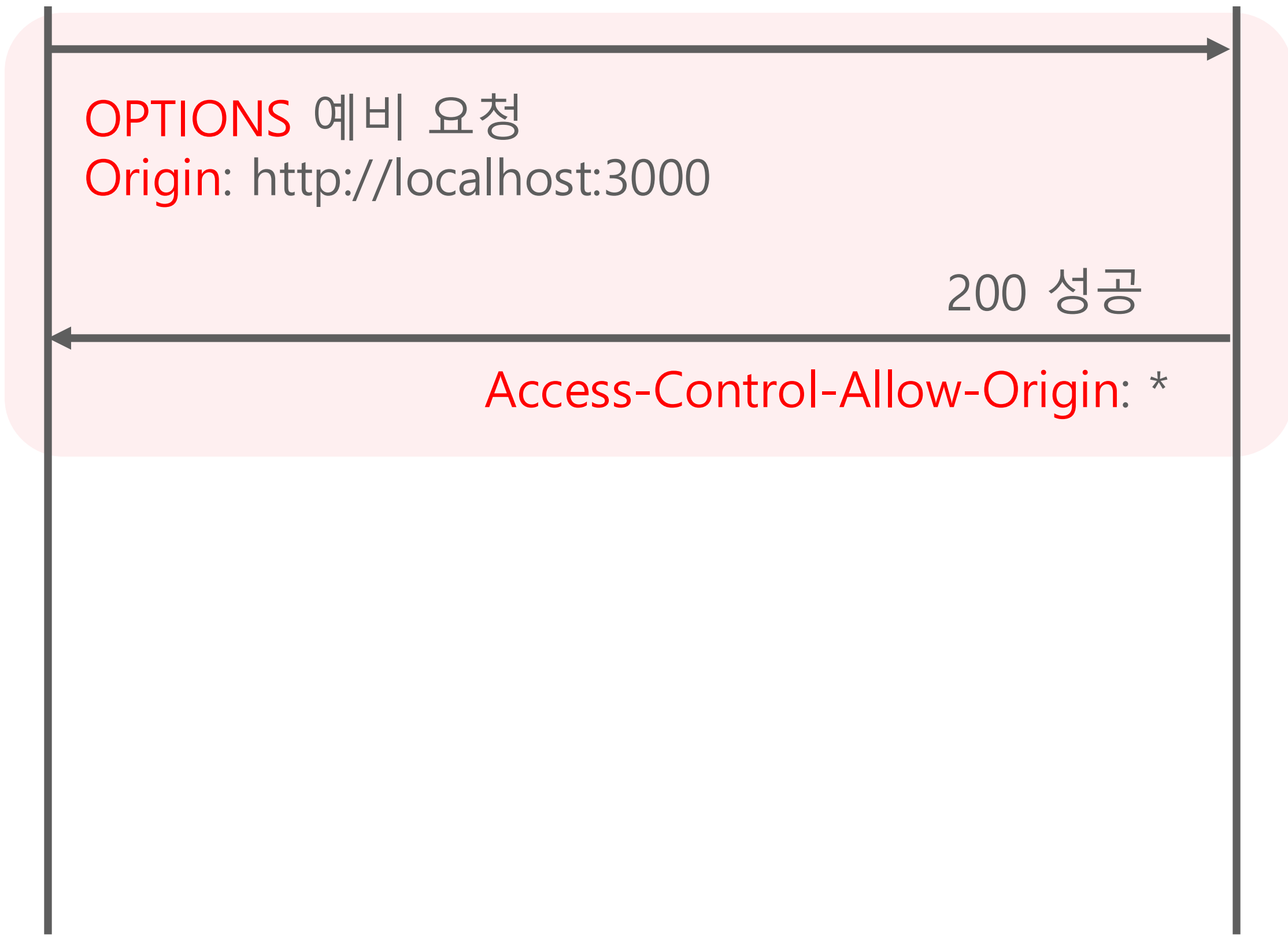




Preflight Request



Server

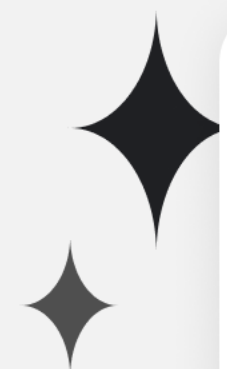


예비 요청 역할 : origin 비교!

Origin $\neq$ Access-Control-Allow-Origin

Name	Method	Status	Protocol	Type	Initiator	Size
index.html	GET	200	http/1.1	docum...		2.5 kB
check.svg	GET	200	h2	svg+xml	index.html	87 B
check.svg	GET	CORS error		fetch	index.html:...	0 B

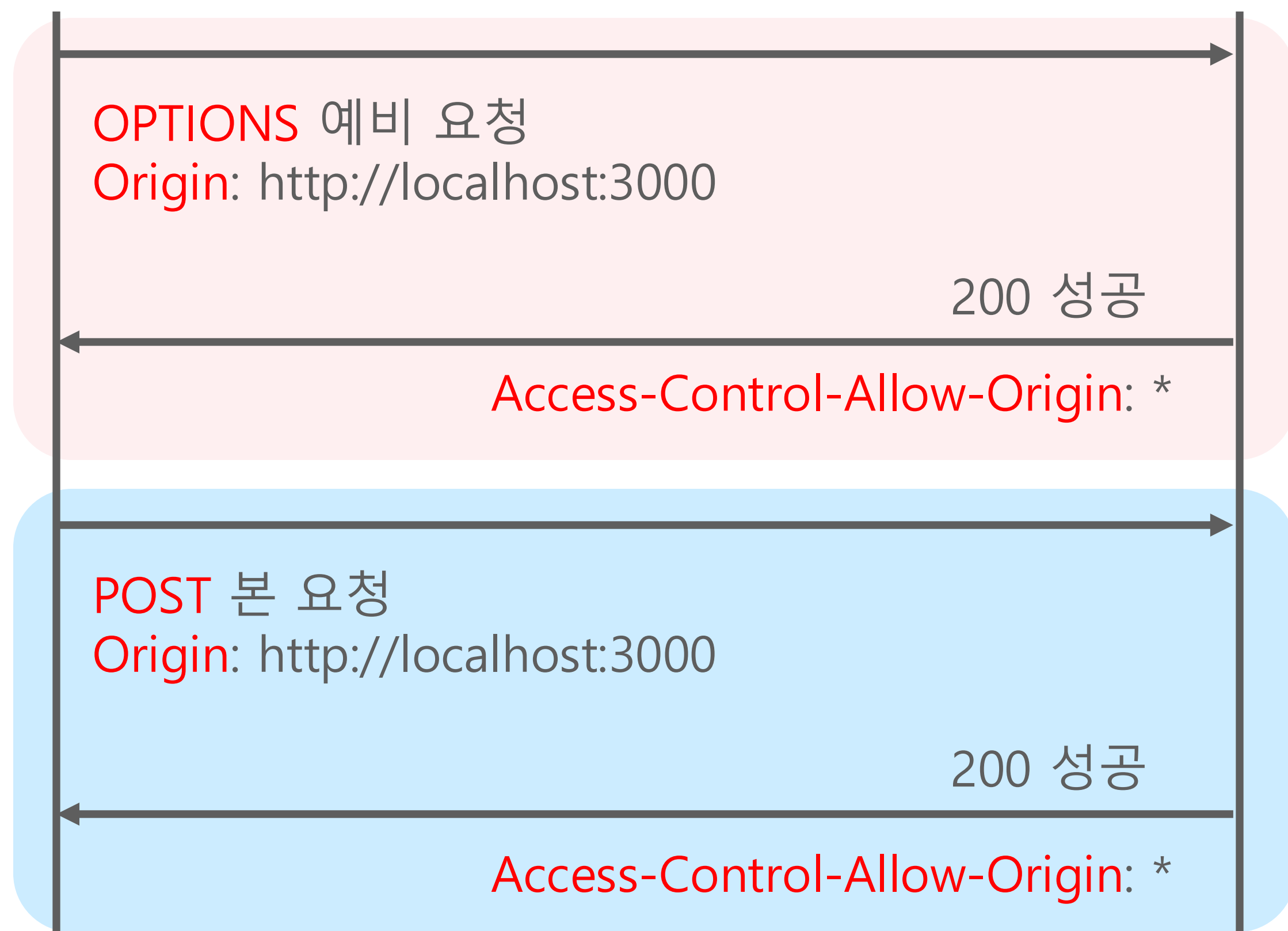




Preflight Request



Server



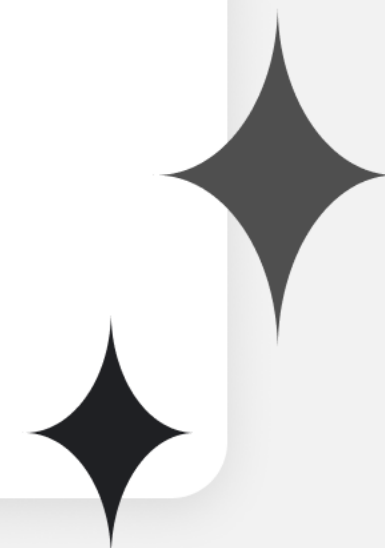
예비 요청 역할 : origin 비교!

Origin $\neq$ Access-Control-Allow-Origin

Name	Method	Status	Protocol	Type	Initiator	Size
index.html	GET	200	http/1.1	docum...		2.5 kB
check.svg	GET	200	h2	svg+xml	index.html	87 B
check.svg	GET	CORS error		fetch	index.html:...	0 B

Origin == Access-Control-Allow-Origin

본요청 실행



❏	getProjectConfig?key=AlzaSyDcadd7...	200	preflight	프리플라이트 🔗
⚡	getProjectConfig?key=AlzaSyDcadd7...	200	xhr	<u>iframe.js:270</u>
❏	accounts:signInWithIdp?key=AlzaSy...	200	preflight	프리플라이트
⚡	accounts:signInWithIdp?key=AlzaSy...	200	fetch	<u>index-C-tlMG12.js:538</u>

▼ 일반

요청 URL:	http://localhost:8080/api
요청 메서드:	OPTIONS
상태 코드:	● 200 OK
원격 주소:	172.217.175.42:443
리퍼러 정책:	no-referrer

▼ 응답 헤더

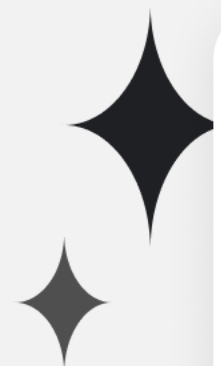
Access-Control-Allow-Headers:	content-type,x-client-version,x-firebase-gmpid
Access-Control-Allow-Methods:	DELETE,GET,HEAD,OPTIONS,PATCH,POST,PUT
Access-Control-Allow-Origin:	*
Access-Control-Max-Age:	3600

Credentialed Request

Preflight Request + Credential

Credential 자격 **인증 정보**

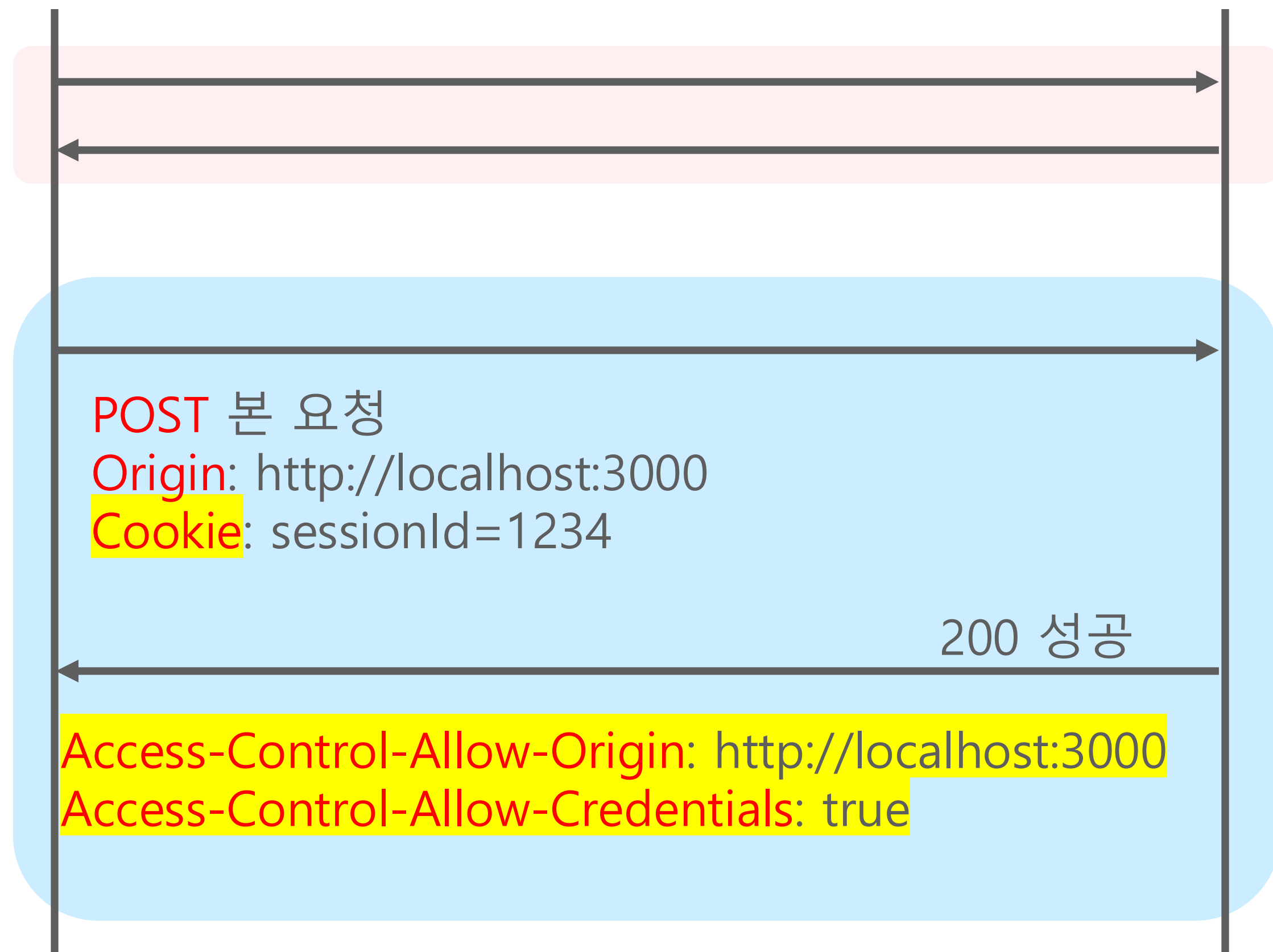
- 쿠키 Cookie
- 세션 Session
- 토큰 Authorization 헤더
- ...



Credentialed Request

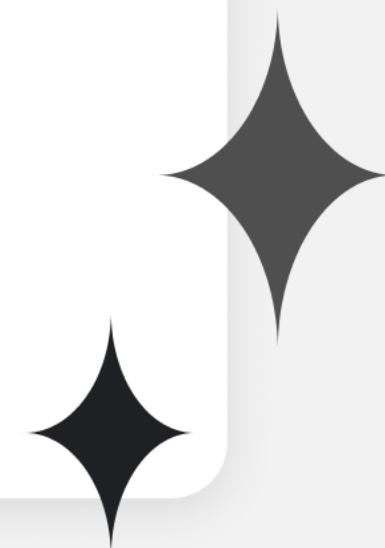


Server



Preflight Request 차이점

1. 요청 헤더 인증 정보 Cookie
2. Access-Control-Allow-Origin: * 불가능
3. 본 요청 응답 헤더 credentials true 옵션



Credential 옵션



```
// fetch 메서드
fetch("https://localhost:8080/api", {
  method: "POST",
  credentials: "include", // 인증 정보 포함
  body: JSON.stringify({
    userId: 1,
  }),
})

// axios 라이브러리
axios.post('https://localhost:8080/api', {
  profile: { username: username, password: password }
}, {
  withCredentials: true // 인증 정보 포함
})
```

```
@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/**")
            .allowedOrigins("https://localhost:3000") // 허용 출처
            .allowCredentials(true); // 인증 정보 포함
    }
}
```

서버에서 구현할 수 없을 때는?

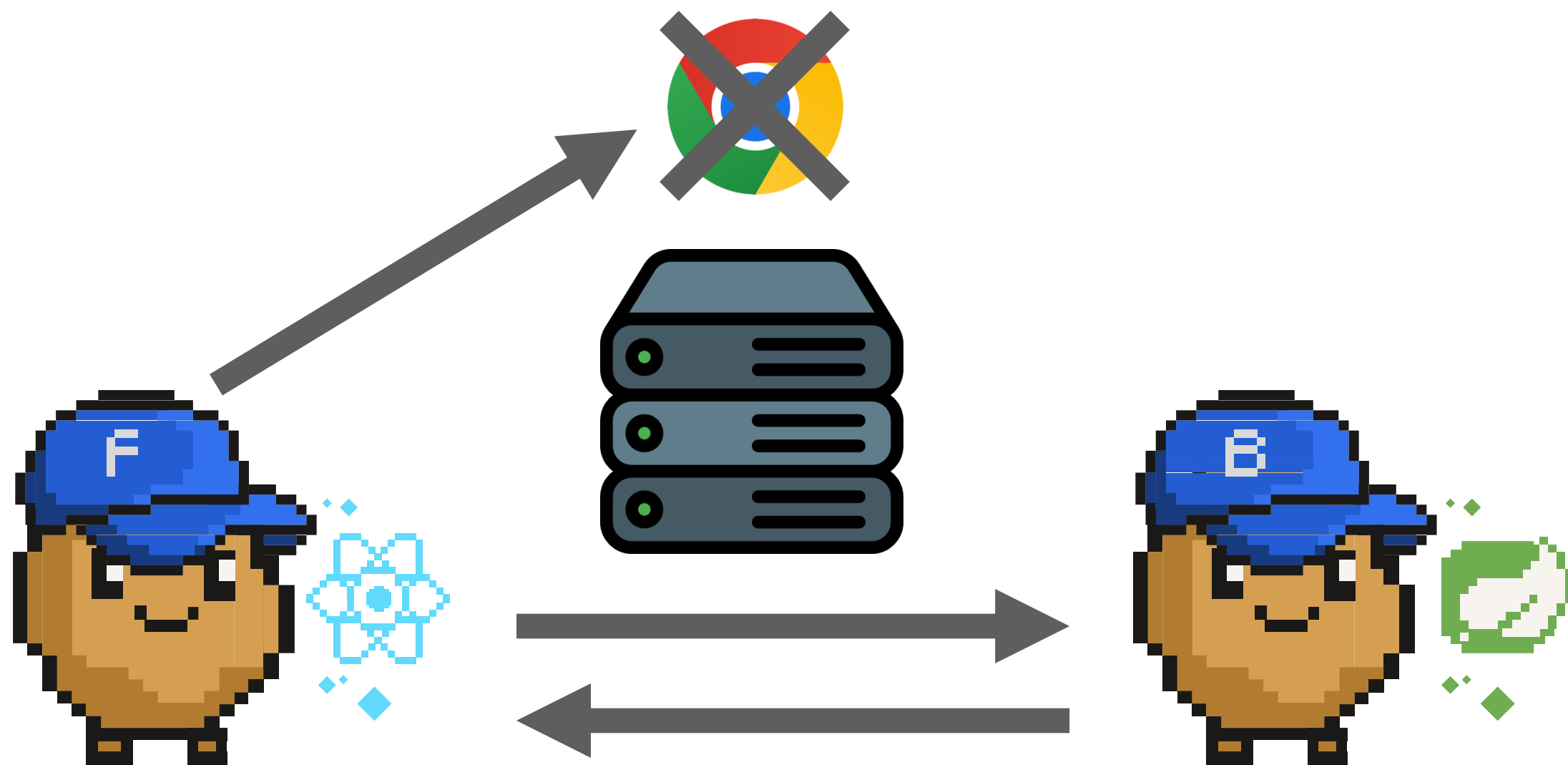
Chrome 확장 프로그램 이용

 **Allow CORS: Access-Control-Allow-Origin**

3.4 ★ (평점 279개)

서버에서 구현할 수 없을 때는?

Proxy 프록시 서버



```
// vite.config.json
export default defineConfig({
  server: {
    proxy: {
      '/api': {
        target: 'http://www.google.com',
        changeOrigin: true,
        rewrite: (path) => path.replace(/^\/api/, ''),
        secure: false,
        ws: true,
      },
    },
  },
});
```